

VÕ TIẾN

Thảo luận kiến thức CNTT trường BK về KHMT(CScience), KTMT(CEngineering)
<https://www.facebook.com/groups/khmt.ktmt.cse.bku>



Kỹ Thuật Lập Trình (Cơ bản và nâng cao C++)

KTILT2 - HK242

TASK 9 Nghệ thuật lập trình - Đa hình và Trừu tượng hóa

Thảo luận kiến thức CNTT trường BK
về KHMT(CScience), KTMT(CEngineering)
<https://www.facebook.com/groups/khmt.ktmt.cse.bku>

Mục lục

1 Đa hình (Polymorphism)	2
1.1 Khái niệm	2
1.2 Đa hình tại thời điểm biên dịch (Compile-time Polymorphism)	2
1.3 Đa hình tại thời điểm chạy (Run-time Polymorphism)	3
2 Khởi tạo đối tượng	4
2.1 Khái niệm	4
2.2 Khởi tạo trong stack đối tượng	4
2.3 Khởi tạo trong heap đối tượng	5
2.4 Khởi tạo thừa kế bằng con trỏ	5
2.5 Ép kiểu	6
2.5.1 Upcasting	6
2.5.2 Downcasting với static_cast	7
2.5.3 Downcasting với static_cast	7
3 Hàm ảo (Virtual Functions)	9
3.1 Hàm ảo cơ bản	9
3.2 Hàm ảo với tham chiếu	10
3.3 Hàm hủy ảo (Virtual Destructor)	10
4 Hàm thuần ảo (Pure Virtual Function)	12
5 Lớp trừu tượng (Abstract Class)	13
6 Lớp cơ sở ảo (Virtual Base Class)	14
7 Truy cập thành viên tĩnh	15
8 Giao diện (Interface)	16



1 Đa hình (Polymorphism)

1.1 Khái niệm

Đa hình là khả năng của một đối tượng hoặc hàm trong C++ để thể hiện nhiều hình thức (forms) khác nhau tùy theo ngữ cảnh. Từ "polymorphism" trong tiếng Hy Lạp nghĩa là "nhiều hình dạng" (poly = nhiều, morph = hình dạng). Trong OOP, đa hình cho phép xử lý các đối tượng của các lớp khác nhau thông qua một giao diện chung.

Đặc điểm:

- Từ "polymorphism" trong tiếng Hy Lạp nghĩa là "nhiều hình dạng" (poly = nhiều, morph = hình dạng).
- Có hai loại chính trong C++:
 - **Đa hình thời gian biên dịch (Compile-time Polymorphism)**: Thông qua **quá tải hàm** và **quá tải toán tử**.
 - **Đa hình thời gian chạy (Runtime Polymorphism)**: Thông qua **hàm ảo** và **kế thừa**.

```
1 class Animal {
2 public:
3     virtual void sound() { // Hàm ảo
4         cout << "Some generic sound" << endl;
5     }
6     virtual ~Animal() {}
7 };
8
9 class Dog : public Animal {
10 public:
11     void sound() override { // Ghi đê
12         cout << "Woof!" << endl;
13     }
14 };
15
16 class Cat : public Animal {
17 public:
18     void sound() override {
19         cout << "Meow!" << endl;
20     }
21 };
22
23 Animal* animal = new Dog();
24 animal->sound(); // Output: Woof! (đa hình thời gian chạy)
25 delete animal;
26 animal = new Cat();
27 animal->sound(); // Output: Meow!
28 delete animal;
```

1.2 Đa hình tại thời điểm biên dịch (Compile-time Polymorphism)

Định nghĩa: Đây là loại đa hình được xác định và giải quyết ngay tại thời điểm chương trình được biên dịch (trước khi chạy). Trình biên dịch sẽ quyết định hàm nào được gọi dựa trên thông tin có sẵn (ví dụ: số lượng hoặc kiểu tham số).

- Được thực hiện thông qua **nạp chồng hàm (function overloading)** hoặc **nạp chồng toán tử (operator overloading)**.



- Ví dụ: Bạn có thể định nghĩa nhiều hàm cùng tên nhưng khác tham số.

```
1 class Example {
2 public:
3     void print(int x) { cout << "Integer: " << x << endl; }
4     void print(double x) { cout << "Double: " << x << endl; }
5 };
6
7 Example obj;
8 obj.print(5);           // Gọi hàm với int
9 obj.print(5.5);        // Gọi hàm với double
```

1.3 Đa hình tại thời điểm chạy (Run-time Polymorphism)

Định nghĩa: Đây là loại đa hình được xác định trong thời gian chương trình chạy. Nó cho phép một giao diện chung (hàm ảo) được gọi, nhưng hành vi cụ thể sẽ phụ thuộc vào đối tượng thực tế đang được tham chiếu.

- Được thực hiện thông qua hàm ảo (virtual functions) và kế thừa (inheritance).
- Cho phép gọi phương thức của lớp con thông qua con trỏ hoặc tham chiếu của lớp cha.

```
1 class Animal {
2 public:
3     virtual void sound() { cout << "Some generic animal sound" << endl; }
4 };
5
6 class Dog : public Animal {
7 public:
8     void sound() override { cout << "Woof Woof" << endl; }
9 };
10
11 Animal* animal = new Dog();
12 animal->sound(); // In ra: "Woof Woof"
13 delete animal;
```



2 Khởi tạo đối tượng

2.1 Khái niệm

Khởi tạo đối tượng là quá trình cấp phát bộ nhớ và thiết lập trạng thái ban đầu cho một đối tượng của một lớp hoặc kiểu dữ liệu trong C++. Quá trình này thường được thực hiện thông qua **hàm tạo** (**constructor**), một hàm thành viên đặc biệt được gọi tự động khi đối tượng được tạo. **Khởi tạo đảm bảo đối tượng ở trạng thái hợp lệ ngay từ khi tồn tại, tránh lỗi do truy cập dữ liệu chưa khởi tạo.**

- Hàm tạo không có kiểu trả về, trùng tên với lớp.
- Nếu không cung cấp hàm tạo, trình biên dịch tạo hàm tạo mặc định (nhưng không khởi tạo thành viên nguyên thủy).
- Liên quan đến kế thừa, đa hình, và quản lý tài nguyên.

```
1 class Point {
2 public:
3     int x, y;
4     Point() : x(0), y(0) {} // Hàm tạo mặc định
5     Point(int xVal, int yVal) : x(xVal), y(yVal) {} // Hàm tạo có tham số
6 };
7
8 int main() {
9     Point p1; // Khởi tạo mặc định
10    Point p2(3, 4); // Khởi tạo có tham số
11    cout << p1.x << "," << p1.y << endl; // Output: 0,0
12    cout << p2.x << "," << p2.y << endl; // Output: 3,4
13    return 0;
14 }
```

2.2 Khởi tạo trong stack đối tượng

- Khởi tạo trên **ngăn xếp (stack)** là khi đối tượng được tạo trong bộ nhớ tự động (automatic storage). Đối tượng tồn tại trong phạm vi khai báo và tự động được hủy khi ra khỏi phạm vi (scope) nhờ cơ chế **RAII** (Resource Acquisition Is Initialization).
- Nhanh hơn khởi tạo trên heap vì không cần cấp phát động, nhưng bị giới hạn bởi kích thước stack (thường vài MB).
- Bộ nhớ được cấp phát tại compile-time, hàm tạo được gọi ngay khi đối tượng được khai báo.
- Hàm hủy được gọi tự động khi đối tượng ra khỏi phạm vi.

```
1 class Box {
2 public:
3     int size;
4     Box(int s) : size(s) { cout << "Box created with size " << size << endl; }
5     ~Box() { cout << "Box destroyed" << endl; }
6 };
7
8 int main() {
9     Box b(10); // Khởi tạo trên stack
10    cout << "Size: " << b.size << endl; // Output: Box created with size 10 \n
11    // Size: 10
12    return 0; // Tự động gọi ~Box() -> Output: Box destroyed
13 }
```



2.3 Khởi tạo trong heap đối tượng

- Khởi tạo trên **đống (heap)** là khi đối tượng được tạo trong bộ nhớ động (dynamic storage) bằng toán tử new. Đối tượng tồn tại cho đến khi được hủy thủ công bằng delete, cho phép kiểm soát vòng đời linh hoạt hơn stack.
- Chậm hơn stack do cấp phát động, nhưng không bị giới hạn kích thước (chỉ giới hạn bởi RAM).
- new cấp phát bộ nhớ trên heap, sau đó gọi hàm tạo với tham số (nếu có), trả về con trỏ.
- delete gọi hàm hủy và giải phóng bộ nhớ.

```
1 class Circle {
2 public:
3     int radius;
4     Circle(int r) : radius(r) { cout << "Circle created with radius " << radius <<
5         → endl; }
6     ~Circle() { cout << "Circle destroyed" << endl; }
7 };
8
9 int main() {
10     Circle* c = new Circle(5); // Khởi tạo trên heap
11     cout << "Radius: " << c->radius << endl; // Output: Circle created with radius
12         → 5 \n Radius: 5
13     delete c; // Gọi ~Circle() -> Output: Circle destroyed
14     return 0;
15 }
```

2.4 Khởi tạo thừa kế bằng con trỏ

- Trong kế thừa, khi khởi tạo đối tượng của lớp dẫn xuất thông qua con trỏ lớp cơ sở, hàm tạo của lớp cơ sở được gọi trước, sau đó đến hàm tạo của lớp dẫn xuất. Điều này đảm bảo đối tượng được khởi tạo đầy đủ theo hệ thống phân cấp.
- Dùng con trỏ hỗ trợ đa hình thời gian chạy (runtime polymorphism), cho phép gọi hàm ảo của lớp dẫn xuất thông qua giao diện lớp cơ sở.
- Khi dùng new Derived(), trình biên dịch gọi chuỗi hàm tạo từ lớp cơ sở đến lớp dẫn xuất.
- Con trỏ lớp cơ sở trỏ đến đối tượng dẫn xuất, nhưng chỉ có thể truy cập thành viên của lớp cơ sở trừ khi ép kiểu.

```
1 class Base {
2 public:
3     Base() { cout << "Base constructor" << endl; }
4     virtual void show() { cout << "Base show" << endl; }
5     virtual ~Base() { cout << "Base destructor" << endl; }
6 };
7
8 class Derived : public Base {
9 public:
10     Derived() { cout << "Derived constructor" << endl; }
11     void show() override { cout << "Derived show" << endl; }
12     ~Derived() override { cout << "Derived destructor" << endl; }
13 };
14
15 int main() {
16     Base* b = new Derived(); // Khởi tạo thừa kế bằng con trỏ
17 }
```



```
17     b->show(); // Output: Base constructor \n Derived constructor \n Derived show
18     delete b; // Output: Derived destructor \n Base destructor
19     return 0;
20 }
```

2.5 Ép kiểu

Ép kiểu (type casting) trong C++ là quá trình chuyển đổi một đối tượng hoặc con trỏ từ kiểu dữ liệu này sang kiểu dữ liệu khác. Trong bối cảnh lập trình hướng đối tượng (OOP), ép kiểu thường được sử dụng để xử lý các mối quan hệ giữa các lớp trong hệ thống phân cấp kế thừa, đặc biệt với **đa hình (polymorphism)**. C++ cung cấp nhiều cách ép kiểu, từ kiểu truyền thống (C-style) đến các toán tử ép kiểu hiện đại (static_cast, dynamic_cast, reinterpret_cast, const_cast), mỗi loại có mục đích và mức độ an toàn riêng.

- **Upcasting:** An toàn, tự động, chuyển từ lớp dẫn xuất lên lớp cơ sở.
- **Downcasting:** Không an toàn trừ khi dùng dynamic_cast, chuyển từ lớp cơ sở xuống lớp dẫn xuất.
- **Cross-casting:** Ép kiểu giữa các lớp không trực tiếp liên quan trong hệ thống phân cấp (thường cần dynamic_cast).
- **Ép kiểu không liên quan đến kế thừa:** Dùng reinterpret_cast hoặc const_cast, ít phổ biến trong OOP.

2.5.1 Upcasting

1. Lý thuyết:

- Upcasting là quá trình chuyển đổi từ lớp dẫn xuất lên lớp cơ sở, luôn an toàn vì lớp dẫn xuất "là một" (is-a) lớp cơ sở theo quan hệ kế thừa.
- Thường không cần toán tử ép kiểu rõ ràng, diễn ra tự động.

2. Cơ chế bên dưới:

- Con trỏ/tham chiếu được điều chỉnh để trỏ đến phần lớp cơ sở của đối tượng, không thay đổi bố trí bộ nhớ.
- Duy trì đa hình nhờ vtable nếu có hàm ảo.

```
1  class Animal {
2  public:
3      virtual void sound() { cout << "Generic sound" << endl; }
4      virtual ~Animal() {}
5  };
6
7  class Dog : public Animal {
8  public:
9      void sound() override { cout << "Woof!" << endl; }
10 };
11
12 int main() {
13     Dog d;
14     Animal* a = &d; // Upcasting tự động
15     a->sound(); // Output: Woof! (đa hình)
16     return 0;
17 }
```



2.5.2 Downcasting với static_cast

1. Lý thuyết:

- Downcasting chuyển từ lớp cơ sở xuống lớp dẫn xuất, không an toàn trừ khi lập trình viên chắc chắn về kiểu thực tế của đối tượng.
 - static_cast kiểm tra mối quan hệ kế thừa tại compile-time, không kiểm tra runtime.
2. **Cơ chế bên dưới:** Điều chỉnh con trỏ dựa trên bố trí bộ nhớ của lớp, không dùng RTTI, nhanh nhưng có thể gây lỗi nếu ép kiểu sai.

```
1 class Base {
2 public:
3     virtual void show() { cout << "Base" << endl; }
4     virtual ~Base() {}
5 };
6
7 class Derived : public Base {
8 public:
9     void show() override { cout << "Derived" << endl; }
10    void extra() { cout << "Extra" << endl; }
11 };
12
13 int main() {
14     Derived d;
15     Base* b = &d; // Upcasting
16     Derived* dPtr = static_cast<Derived*>(b); // Downcasting
17     dPtr->extra(); // Output: Extra
18     return 0;
19 }
```

Nếu b không thực sự trỏ đến Derived, hành vi không xác định (undefined behavior).

2.5.3 Downcasting với dynamic_cast

1. Lý thuyết:

- dynamic_cast là cách ép kiểu an toàn trong OOP, kiểm tra kiểu thực tế của đối tượng tại runtime nhờ RTTI. Nó trả về nullptr (cho con trỏ) hoặc ném ngoại lệ (cho tham chiếu) nếu ép kiểu thất bại.
 - Yêu cầu lớp có ít nhất một hàm ảo.
2. **Cơ chế bên dưới:** Dùng RTTI và vtable để xác định kiểu thực tế, chậm hơn static_cast nhưng an toàn.

```
1 class Base {
2 public:
3     virtual void show() { cout << "Base" << endl; }
4     virtual ~Base() {}
5 };
6
7 class Derived : public Base {
8 public:
9     void show() override { cout << "Derived" << endl; }
10    void extra() { cout << "Extra" << endl; }
11 };
```




```
12
13 int main() {
14     Base* b = new Derived();
15     Derived* d = dynamic_cast<Derived*>(b); // Downcasting an toàn
16     if (d) {
17         d->extra(); // Output: Extra
18     }
19
20     Base* b2 = new Base();
21     Derived* d2 = dynamic_cast<Derived*>(b2); // Thất bại
22     if (!d2) {
23         cout << "Cast failed" << endl; // Output: Cast failed
24     }
25     delete b;
26     delete b2;
27     return 0;
28 }
```



3 Hàm ảo (Virtual Functions)

Hàm ảo là một cơ chế trong C++ cho phép thực hiện **đa hình thời gian chạy (runtime polymorphism)** trong lập trình hướng đối tượng (OOP). Khi một hàm được khai báo là virtual trong lớp cơ sở (base class), nó có thể được **ghi đè (override)** bởi các lớp dẫn xuất (derived classes), và lời gọi hàm sẽ được xác định tại thời điểm chạy dựa trên kiểu thực thể của đối tượng, thay vì kiểu của con trỏ/tham chiếu.

Đặc điểm:

- Dùng từ khóa virtual trong khai báo hàm ở lớp cơ sở.
- Liên quan chặt chẽ đến kế thừa và đa hình, cho phép một giao diện chung gọi các hành vi khác nhau.
- C++ dùng **bảng hàm ảo (vtable)** để quản lý các lời gọi hàm ảo tại runtime.

```
1 class Animal {
2 public:
3     virtual void sound() { // Hàm ảo
4         cout << "Generic sound" << endl;
5     }
6     virtual ~Animal() {}
7 };
8
9 class Dog : public Animal {
10 public:
11     void sound() override { // Ghi đè
12         cout << "Woof!" << endl;
13     }
14 };
15
16 Animal* a = new Dog();
17 a->sound(); // Output: Woof! (đa hình thời gian chạy)
18 delete a;
```

Lưu ý quan trọng

- **Không có virtual, không đa hình:** Nếu hàm không được khai báo virtual, lời gọi dựa trên kiểu con trỏ/tham chiếu, không phải kiểu thực thể.
- **Hiệu suất:** Hàm ảo dùng vtable, tăng chi phí bộ nhớ (8 byte/con trỏ vtable trên 64-bit) và thời gian tra cứu so với hàm thông thường.
- **Từ khóa override:** Dùng override để đảm bảo hàm ghi đè đúng hàm ảo, tránh lỗi khi chữ ký không khớp.
- **Hàm hủy ảo:** Luôn khai báo hàm hủy ảo trong lớp cơ sở nếu có kế thừa để tránh rò rỉ tài nguyên.

3.1 Hàm ảo cơ bản

Khai báo hàm với từ khóa virtual trong lớp cơ sở, cung cấp triển khai mặc định (hoặc không).

- Trình biên dịch tạo một vtable cho lớp Shape, lưu trữ địa chỉ của draw().
- Khi Circle ghi đè, vtable của Circle trỏ đến phiên bản draw() của nó.
- Lời gọi qua con trỏ s tra cứu vtable để xác định hàm đúng.



```
1 class Shape {
2 public:
3     virtual void draw() { // Hàm ảo với triển khai mặc định
4         cout << "Drawing a generic shape" << endl;
5     }
6     virtual ~Shape() {}
7 };
8
9 class Circle : public Shape {
10 public:
11     void draw() override { // Ghi đè
12         cout << "Drawing a circle" << endl;
13     }
14 };
15
16 Shape* s = new Circle();
17 s->draw(); // Output: Drawing a circle
18 delete s;
```

3.2 Hàm ảo với tham chiếu

Đa hình qua tham chiếu cũng hoạt động tương tự như con trỏ.

```
1 class Base {
2 public:
3     virtual void show() { cout << "Base" << endl; }
4     virtual ~Base() {}
5 };
6
7 class Derived : public Base {
8 public:
9     void show() override { cout << "Derived" << endl; }
10 };
11
12 void display(Base& obj) {
13     obj.show(); // Đa hình
14 }
15
16 Derived d;
17 display(d); // Output: Derived
```

3.3 Hàm hủy ảo (Virtual Destructor)

Hàm hủy cần được khai báo ảo trong lớp cơ sở để đảm bảo hủy đúng các lớp dẫn xuất khi dùng con trỏ lớp cơ sở. Không dùng virtual: Chỉ Base() được gọi, gây rò rỉ tài nguyên nếu Derived cấp phát động.

```
1 class Base {
2 public:
3     virtual void show() { cout << "Base" << endl; }
4     virtual ~Base() { cout << "Base destructor" << endl; }
5 };
6
```



```
7 class Derived : public Base {
8 public:
9     void show() override { cout << "Derived" << endl; }
10    ~Derived() override { cout << "Derived destructor" << endl; }
11 };
12
13 Base* b = new Derived();
14 delete b; // Gọi cả ~Derived() và ~Base() nhờ virtual
15 // Output:
16 // Derived destructor
17 // Base destructor
```



4 Hàm thuần ảo (Pure Virtual Function)

Hàm thuần ảo là một loại hàm ảo đặc biệt trong C++, được khai báo trong lớp cơ sở (base class) với cú pháp `= 0`, không cung cấp triển khai (implementation). Lớp chứa hàm thuần ảo trở thành **lớp cơ sở trừu tượng (abstract base class - ABC)** và không thể khởi tạo trực tiếp. Hàm thuần ảo buộc các lớp dẫn xuất (derived classes) phải ghi đè (override) để cung cấp triển khai cụ thể, hỗ trợ mạnh mẽ đa hình thời gian chạy (runtime polymorphism).

- Cú pháp: `virtual return_type function_name() = 0;`
- Lớp chứa hàm thuần ảo là trừu tượng, không thể tạo đối tượng từ nó.
- Dùng để định nghĩa giao diện (interface) chung mà các lớp dẫn xuất phải tuân theo.

```
1 class Vehicle {
2 public:
3     virtual void move() = 0; // Hàm thuần ảo
4     virtual void stop() {    // Hàm ảo có triển khai
5         cout << "Vehicle stops" << endl;
6     }
7     void honk() {           // Hàm thường
8         cout << "Beep!" << endl;
9     }
10    virtual ~Vehicle() {}
11 };
12
13 class Car : public Vehicle {
14 public:
15     void move() override {
16         cout << "Car drives" << endl;
17     }
18     void stop() override { // Ghi đè tùy chọn
19         cout << "Car brakes" << endl;
20     }
21 };
22
23 Vehicle* v = new Car();
24 v->move(); // Output: Car drives
25 v->stop(); // Output: Car brakes
26 v->honk(); // Output: Beep!
27 delete v;
```

Lưu ý quan trọng

- **Lớp trừu tượng:** Lớp có hàm thuần ảo không thể khởi tạo trực tiếp (Animal a; sẽ lỗi).
- **Bắt buộc ghi đè:** Nếu lớp dẫn xuất không ghi đè hàm thuần ảo, nó vẫn là lớp trừu tượng.
- **Hiệu suất:** Giống hàm ảo thông thường, dùng vtable, chi phí nhỏ (8 byte/con trỏ vtable trên 64-bit).
- **Hàm hủy ảo:** Lớp trừu tượng vẫn nên có hàm hủy ảo để đảm bảo hủy đúng.



5 Lớp trừu tượng (Abstract Class)

Lớp trừu tượng trong C++ là một khái niệm trung tâm của lập trình hướng đối tượng (OOP), đại diện cho một mức độ **trừu tượng hóa (abstraction)** cao trong việc thiết kế phần mềm. Nó là một lớp không hoàn chỉnh (incomplete) về mặt triển khai, chứa ít nhất một **hàm thuần ảo (pure virtual function)**, được khai báo bằng cú pháp `virtual function() = 0`. Điều này biến lớp thành một **lớp cơ sở trừu tượng (Abstract Base Class - ABC)**, không thể khởi tạo trực tiếp, mà chỉ tồn tại để cung cấp một **giao diện (interface)** hoặc **khung hành vi (behavioral skeleton)** cho các lớp dẫn xuất.

- Lớp trừu tượng là một biểu hiện của nguyên tắc **trừu tượng hóa dữ liệu (data abstraction)** trong OOP, nơi chi tiết triển khai được ẩn đi, chỉ để lại giao diện cần thiết.
- Nó liên kết chặt chẽ với **đa hình thời gian chạy (runtime polymorphism)**, cho phép các lớp dẫn xuất ghi đè (override) hành vi của hàm thuần ảo, từ đó tạo ra các thực thể cụ thể (concrete) từ một khuôn mẫu chung.
- Trong mô hình bộ nhớ của C++, lớp trừu tượng không khác biệt về mặt cấu trúc so với lớp thông thường, nhưng trình biên dịch áp đặt hạn chế khởi tạo dựa trên sự hiện diện của hàm thuần ảo.

```
1 class Animal {
2 public:
3     virtual void sound() = 0; // Hàm thuần ảo
4     virtual ~Animal() {}
5 };
6
7 class Dog : public Animal {
8 public:
9     void sound() override { cout << "Woof!" << endl; }
10 };
11
12 // Animal a; // Lỗi: không thể khởi tạo lớp trừu tượng
13 Animal* a = new Dog();
14 a->sound(); // Output: Woof!
15 delete a;
```



6 Lớp cơ sở ảo (Virtual Base Class)

Lớp cơ sở ảo là một cơ chế trong C++ được sử dụng để giải quyết **vấn đề kim cương (diamond problem)** trong kế thừa nhiều lớp (multiple inheritance). Khi một lớp được khai báo là cơ sở ảo bằng từ khóa `virtual` trong khai báo kế thừa, nó đảm bảo rằng chỉ có **một bản sao duy nhất** của lớp cơ sở đó tồn tại trong hệ thống phân cấp, bất kể nó được kế thừa qua bao nhiêu đường dẫn.

- Dùng từ khóa `virtual` trước từ khóa truy cập (`public`, `protected`, `private`) khi kế thừa.
- Giải quyết vấn đề trùng lặp dữ liệu và xung đột tên trong kế thừa nhiều lớp.
- Liên quan đến việc quản lý bộ nhớ và truy cập thành viên trong hệ thống phân cấp phức tạp.

```
1 class Base {
2 public:
3     int value = 10;
4     void show() { cout << "Base: " << value << endl; }
5 };
6
7 class B : virtual public Base {};
8 class C : virtual public Base {};
9 class D : public B, public C {};
10
11 D d;
12 d.show(); // Output: Base: 10 (chỉ một bản sao của Base)
```

Phần 5: Lưu ý quan trọng

1. **Khởi tạo lớp cơ sở ảo:** Lớp dẫn xuất cuối cùng (như D) chịu trách nhiệm gọi hàm tạo của lớp cơ sở ảo, bỏ qua các lớp trung gian.
2. **Hiệu suất:** Kế thừa ảo phức tạp hơn kế thừa thông thường, thêm chi phí nhỏ để quản lý một bản sao duy nhất (thường qua con trỏ nội bộ).
3. **Không cần thiết trong kế thừa đơn:** Chỉ dùng lớp cơ sở ảo khi có kế thừa nhiều lớp và nguy cơ trùng lặp.
4. **Truy cập thành viên:** Không còn mơ hồ khi truy cập thành viên của lớp cơ sở ảo, nhưng cần chú ý nếu kết hợp với kế thừa không ảo.
5. **Hàm hủy ảo:** Lớp cơ sở ảo vẫn nên có hàm hủy ảo nếu dùng đa hình để đảm bảo hủy đúng.



7 Truy cập thành viên tĩnh

Trong C++, **thành viên tĩnh (static members)** là các thành viên (biến hoặc hàm) của một lớp được khai báo với từ khóa `static`. Chúng thuộc về **lớp (class)** chứ không thuộc về **đối tượng (object)** cụ thể, nghĩa là chỉ tồn tại một bản sao duy nhất trong suốt chương trình, bất kể có bao nhiêu đối tượng của lớp được tạo ra. **Truy cập thành viên tĩnh** là cách sử dụng các thành viên này thông qua tên lớp hoặc (ít phổ biến hơn) qua đối tượng.

1. Lý thuyết cơ bản:

- Thành viên tĩnh được liên kết tĩnh (statically bound) tại thời điểm biên dịch, không phụ thuộc vào trạng thái của đối tượng hay cơ chế đa hình thời gian chạy (runtime polymorphism).
- Biến tĩnh (static variables) được lưu trong vùng nhớ tĩnh (static storage), tồn tại suốt vòng đời chương trình.
- Hàm tĩnh (static functions) không có quyền truy cập đến con trỏ `this`, vì chúng không hoạt động trên một đối tượng cụ thể.

2. Đặc điểm:

- Truy cập qua toán tử phạm vi (`::`) với tên lớp (ví dụ: `ClassName::staticMember`).
- Có thể truy cập mà không cần khởi tạo đối tượng.
- Không tham gia vào đa hình (không thể khai báo `virtual`).

```
1 class Counter {
2 public:
3     static int instances; // Khai báo biến tĩnh
4     Counter() { instances++; }
5     ~Counter() { instances--; }
6 };
7
8 int Counter::instances = 0; // Định nghĩa biến tĩnh
9
10 int main() {
11     Counter c1, c2;
12     cout << Counter::instances << endl; // Output: 2
13     return 0;
14 }
```

```
1 class Math {
2 public:
3     static int add(int a, int b) {
4         return a + b;
5     }
6 };
7
8 int main() {
9     int sum = Math::add(3, 4);
10    cout << sum << endl; // Output: 7
11    return 0;
12 }
```




8 Giao diện (Interface)

Trong Java, **giao diện (interface)** là một cấu trúc cú pháp riêng biệt, được khai báo bằng từ khóa `interface`, dùng để định nghĩa một **hợp đồng (contract)** về hành vi mà các lớp triển khai (implementing classes) phải tuân theo. Giao diện chỉ chứa các **phương thức trừu tượng (abstract methods)** (trước Java 8) hoặc kết hợp với các phương thức mặc định (default methods) và phương thức tĩnh (static methods) từ Java 8 trở đi. Nó là công cụ chính để thực hiện **đa hình (polymorphism)** và **trừu tượng hóa (abstraction)** trong Java.

Lý thuyết cơ bản:

- Giao diện là một **kiểu tham chiếu (reference type)**, tương tự như lớp, nhưng không thể khởi tạo trực tiếp (tức là không có `new Interface()`).
- Trước Java 8, giao diện chỉ chứa các phương thức trừu tượng công khai (`public abstract`) và hằng số (`public static final`). Từ Java 8, nó được mở rộng để hỗ trợ triển khai mặc định, tăng tính linh hoạt.
- Giao diện hỗ trợ **kế thừa nhiều (multiple inheritance)** trong Java, khắc phục hạn chế của lớp (Java không cho phép kế thừa nhiều lớp).

```
interface Printable {  
    void print(); // Phương thức trừu tượng  
}  
  
class Document implements Printable {  
    public void print() {  
        System.out.println("Printing document");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Printable p = new Document();  
        p.print(); // Output: Printing document  
    }  
}
```